ForecastLabAI — 2-minute interview pitch and prep checklist

Problem

Have a ready spoken answer for "tell me about a project" and know what to study first.

Instant solution

```
One of the projects I'd highlight is ForecastLabAI, an end-to-end retail demand
forecasting and AI-assisted decision-support application.

The backend is Python with FastAPI and Pydantic, using PostgreSQL, SQLAlchemy and
Alembic for persistence and migrations. The frontend is React and TypeScript, and
the whole stack runs locally or in Docker containers.

The ML pipeline covers data ingestion, feature engineering, model training,
time-series backtesting, model selection and a model registry. I work with Pandas,
NumPy and scikit-learn, plus optional LightGBM and XGBoost. A key concern is
temporal data leakage, so the pipeline uses lagged, shifted features and
chronological validation windows.

On the generative AI side there are PydanticAI agents and retrieval-augmented
generation. The RAG subsystem stores embeddings in PostgreSQL with pgvector and
returns evidence-grounded answers with citations. The experiment agent runs
forecasting evaluations through typed tools, and sensitive actions such as
changing a deployment alias need human approval.

The project also has automated tests, static type checking, CI workflows,
artifact tracking with integrity hashes, and containerized execution. I chose a
modular monolith: clear component boundaries without distributed-system overhead.

What I value most is that it connects ML models with what makes them useful:
APIs, data quality, reproducibility, evaluation, controlled automation and a
user-facing application.
```

Prep checklist (in this order)

- [ ] Time-series forecasting and backtesting — leakage, windows, baselines, metrics
- [ ] Python and scikit-learn — estimator interface, fit/predict, Pandas transforms, tree models
- [ ] Agentic AI with PydanticAI — tool calling, DI, validated outputs, approval
- [ ] RAG and embeddings — chunking, vectors, similarity retrieval, citations
- [ ] FastAPI and PostgreSQL — validation, async sessions, SQLAlchemy
- [ ] MLOps and deployment — registry, artifact integrity, CI/CD, scaling trade-offs

Why it works / gotchas

- About 2 minutes spoken. Say "I" only for parts you built yourself.

- Strongest topics: leakage-safe forecasting, systematic evaluation, typed agent tools, RAG, reproducibility, deliberate trade-offs.
- Don't add Kubernetes, cloud or auth — see DevOps.